

Intelligent Probabilistic Forecasts of Day-Ahead Electricity Prices in a Highly Volatile Power Market

Behrouz Banitalebi
Dept. of Supply Chain Management
University of Manitoba
Winnipeg, Canada
banitalb@myumanitoba.ca

Srimantoora S. Appadoo
Dept. of Supply Chain Management
University of Manitoba
Winnipeg, Canada
ss.appadoo@umanitoba.ca

Yuvraj Gajpal
Dept. of Supply Chain Management
University of Manitoba
Winnipeg, Canada
yuvraj.gajpal@umanitoba.ca

Aerambamoorthy Thavaneswaran
Dept. of Statistics
University of Manitoba
Winnipeg, Canada
Aerambamoorthy.Thavaneswaran@umanitoba.ca

Abstract—Electricity price forecasting plays an important role in decision making on bidding strategies of selling and buying electricity. This paper computes one day-ahead (DA) quantile forecasts of electricity prices in a highly volatile market by applying regression models to a pool of point forecasts. Three data-driven forecasting methods are implemented to generate DA point forecasts of the Ontario market's electricity prices. In order to generate the three sets of point forecasts, we use: i) the triple exponential smoothing (TES) method, ii) a neural network (NN) that combines layers of Convolutional neurons and gradient recurrent units (GRU), iii) an extreme gradient boosted (XGB) non-linear regression approach. Performance of the three models compared against a benchmark which considers the forecast of electricity prices as the average price of the same hour and day during the last four weeks. The TES method decreases the mean absolute error (MAE) of the benchmark model from 10.29 to 9.42. The Convolutional GRU (ConvGRU) model and XGB regression also reduce the MAE to 8.20 and 7.06, respectively. Finally, quantile regression averaging (QRA) is applied to the pool of point forecasts obtained by TES, ConvGRU, and XGB methods to compute DA quantile forecasts of electricity prices. Moreover, the QRA method is further developed in this work by employing gradient boosting non-linear regression (GBR). Our analysis using real data reveals that the GBR method provides more reliable quantiles as well as tighter prediction intervals with smaller forecasting errors than QRA.

Index Terms—Quantile Forecasts, ConvGRU Neural Networks, XGB/GB Regression, Day-Ahead Electricity Prices.

I. INTRODUCTION

Deregulated structures of electricity supply chains have provided opportunities for power producers to compete in the electricity market. Compare to the regulated markets, the deregulated markets give customers more control over decision-making and provide renewable power producers more opportunities to participate in power production. Consequently, fluctuations in electricity prices in deregulated markets are much higher with larger volatility than the regulated markets. As a result, electricity price forecasting (EPF) plays an important role in electricity trading market. Recently, there has been a growing interest in studying probabilistic electricity

price forecasting (PEPF) as it provides more information about future electricity prices than the point forecasts.

Neural networks have been one of the dominant approaches in the PEPF context. In 2003 the research on PEPF began when prediction intervals (PI) of electricity prices were derived by a cascade neural network model [1]. Later, [2] improved the cascade NN model in [1] by adding a U-D factorization method within a decoupled extended Kalman filter. Reference [3] developed a hybrid PI forecasting method, including bootstrapped neural networks and GARCH model. It is claimed that the hybrid method generates narrow PI with an extensive coverage probability. In a follow-up paper, [4], used delta and bootstrap methods to derive PIs and concluded that while NN prediction errors are relatively large, bootstrap PIs are narrower than delta PIs. An extreme learning machine (ELM) method was used to train NN in [5]. Moreover, the bootstrap method was used in [5] to calculate the PIs of the future electricity prices of the Australian electricity market. In a related work, [6], the ELM method was used in a hybrid structure to compute point forecasts of half-hourly Australian electricity prices and a bootstrap based NN calculated PIs.

PEPF attracted lots of attention in 2014 when 581 participants from 61 countries participated in the Global Energy Forecasting Competition (GEFCom2014). The competition focused on probabilistic forecasts of electricity load and price, as well as renewable energy. The winning team [7] developed a novel model for PEPF based on quantile regression for a generalized additive model (GAM). Compared with some of the statistical and machine learning methods, the model proposed in [7] generates more accurate PEPF. Furthermore, as announced by the members of the winning team, their proposed GAM model can be used to interpret the effect of electricity demand (the explanatory variable) on the electricity price (response variable). The second winning team [9] in GEFCom2014 competition extended the QRA method [8] and proposed a hybrid model for PEPF. The proposed model is not only accurate but also flexible in a way that expert's opinions

on future electricity prices can be incorporated in forecasting of the DA electricity prices. Reference [10], which is ranked 3rd in the competition, uses a simple multilayer perceptron (MLP) with a single hidden layer. The system and zonal electricity load, as well as their squared values, are considered as input data. The input layer was connected to five sigmoid neurons, and a linear neuron generates the point forecasts of electricity prices for the next 24 hours. The model is trained based on the hourly electricity load for 13 previous days. In order to compute PEPF, the forecasting errors were assumed to have a normal distribution in [10].

Recently, [11] showed that taking an average of a predictive model's electricity price forecasts over different calibration windows generates more accurate forecasts than choosing the optimal window sizes. The idea of averaging across various calibration windows is extended by QRA and quantile regression machine (QRM) algorithm to the point forecasts of the DA electricity prices derived by different window sizes [12].

The main contribution of our paper is employing non-linear regression methods such as GBR which results in a more reliable quantile forecasts, with narrower PIs and smaller forecasting errors than the QRA. For our data analysis, we download the historical electricity prices and demand data with hourly resolution from Independent System Operator of Ontario website (ieso.ca). The hourly meteorological data such as weather temperature, solar irradiance, wind speed/direction, and humidity are collected from the government of Canada website (climate.weather.gc.ca) to compute DA forecasts of electricity demand. Moreover, the last comparison study for analysing volatility of electricity returns of Ontario power market has been done in 2007 in [13]. We compare the electricity returns volatility of three electricity markets: Ontario, Pennsylvania-Jersey-Maryland (PJM), and Nord Pool, by using the recently proposed data-driven method for volatility forecasts [14].

The rest of our paper is organized in six sections in which, section II examines volatility of the Ontario electricity market. Section III applies decision tree regression methods to provide DA forecasts of electricity demand in Ontario. Section IV describes artificial neural networks (ANN) models for time series forecasting. Section V implements three different methods to create a pool of point forecasts of DA electricity prices. Derivation and evaluation of quantile forecasts of DA electricity prices are presented in section VI. Finally, section VII concludes the paper.

II. VOLATILITY ANALYSIS

The volatility of an electricity price series refers to the degree of variation of the electricity price over time. In a deregulated market, the price of electricity depends on buyers' and suppliers' decisions at each hour. Besides, the failure of power units or congestion in transmission lines result in unexpected fluctuations in electricity prices. Overall, electricity supply and demand seem to have a substantial impact on electricity prices. In order to validate the statement, we choose the 2019 hourly Ontario electricity price (HOEP) and demand

data. Table I presents some descriptive statistics of the data including mean and standard deviation (Std), minimum and maximum of HOEP, as well as three important quartiles. It seems that while most electricity prices vary between 3.00 and 15.00 CAD/MWh, price spikes cause a relatively high volatility (standard deviation) of the data. We use the Inter-Quartile Range (IQR) to define the threshold and separate regular prices from the spikes. IQR is a descriptive statistics and equals the difference between upper (Q_3) and lower (Q_1) quartiles. In this work, we define spike prices as the samples in the data set that fall below $Q_1 - 1.7 \times IQR$ or above $Q_3 + 1.7 \times IQR$. Based on the definition, 1.15% of the samples in the 2019 HOEP data set are the spike prices.

TABLE I
DESCRIPTIVE STATISTICS OF 2019 ONTARIO ELECTRICITY PRICES

Mean	Std	Min	Q_1	Median	Q_3	Max
16.72	23.55	-59.29	3.02	14.36	25.56	1028.52

A. Determinants of the Price Fluctuations

Our data analysis of the Ontario market's electricity prices shows that there is a correlation of almost 0.40 between hourly Ontario electricity price (HOEP) and demand. Moreover, the correlation between wind power generation and electricity prices in 2019 is almost -0.07 . The impacts of electricity demand and renewable supply on HOEPs are illustrated in Fig. 1. At first glance, it seems that impacts of renewable power plants such as wind power units on electricity prices are negligible. However, it should be noted that almost 80% of the spike prices occur when the total output of wind power units is less than 1500 MW. It could be interpreted as the situations in which wind power plants are scheduled to produce a certain amount of energy but unexpected incidents happen. In fact, the renewable units are scheduled based on DA weather forecasts. If unpredictable weather changes occur and renewable plants' production goes above or below what is expected, spike prices will take place. The recent Texas power crisis in February 2021 indicates the impact of renewable power supplies on electricity prices. The severe winter storm affected the wind and solar power plants in Texas and dropped the power supply dramatically.

The impact of date and time on electricity prices which is shown in Fig.2 is a consequence of the relationship between electricity price and demand. Fig. 2 provides an intuition for temporal patterns exist in 2010-2019 HOEP data set. The time series plots with different scales (hourly/weekly/monthly) show that electricity has higher price in Winter and Summer months than in other seasons. Moreover, electricity costs most at the beginning of the week (Monday) and drops toward the end of the week with the lowest price on Sundays. Within the day, the electricity prices have two peaks in the morning and evening. The impact of time of the day on electricity prices could decrease the validity of volatility analysis. Dividing the hourly time series of the electricity price into 24 subsets is

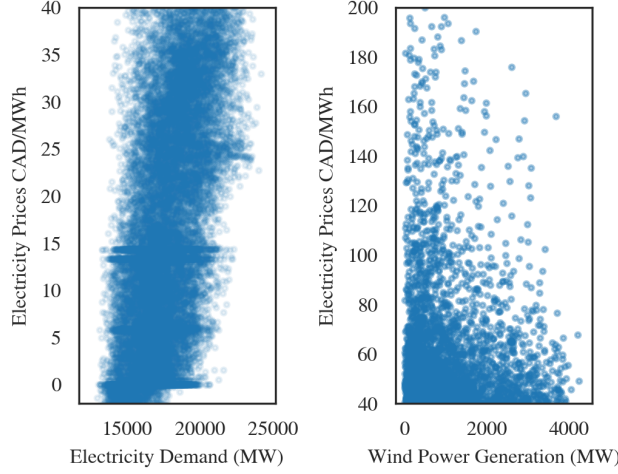


Fig. 1. Effects of load and renewable generations on electricity price.

proposed as a solution in [13]. Authors of [13] calculated hourly volatility of electricity returns for the Ontario, PJM, and New England Markets and concluded that Ontario was the most volatile market in 2007 among other the two markets. In the following, we compute the hourly volatility of the Ontario market's electricity returns to determine whether the Ontario market is still highly volatile.

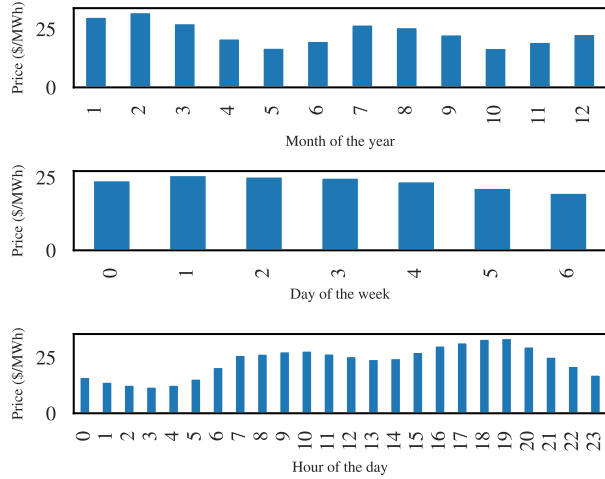


Fig. 2. Effects of time and date on electricity prices.

B. Volatility of Hourly Electricity Returns

In order to compute the volatility of hourly electricity returns, we divide 2019 hourly electricity prices into 24 time series; each has 365 elements. Mathematically, the subsets are shown by $p_{t,h}$ in which $t \in [1, 365]$ and $h \in [1, 24]$. Following [13], arithmetic returns of electricity are obtained as

$$r_{t,h} = p_{t,h} - p_{t-1,h}$$

and used to calculate the sign correlation of the return time series. The idea of sign correlation of time series is proposed in [14], and for the electricity returns is defined as

$$\hat{\rho}_h = \text{Corr}(r_{t,h} - \bar{r}_h, \text{sign}(r_{t,h} - \bar{r}_h)). \quad (1)$$

In eq. (1), $\hat{\rho}_h$ and $\bar{r}_h$ are sample sign correlation and mean of electricity returns at hour h , respectively. Also, $\text{Corr}(\cdot)$ refers to the correlation function. It can be shown that if the electricity return series have a symmetric distribution with finite mean $\bar{r}_h$ and variance $\sigma_{r_h}^2$ then the mean observed hourly volatility, $\bar{s}_h$, is

$$\bar{s}_h = \frac{1}{N_t^s - 1} \sum_{t=2}^{N_t^s} \frac{|r_{t,h} - \bar{r}_h|}{\hat{\rho}_h}.$$

The 2019 HOEPs are considered as testing data in which there are N_t samples. We divide the whole testing data into 24 subsets and each of them has N_t^s samples. Fig.3 shows the mean hourly volatility of electricity returns of Ontario, PJM, and Nord Pool markets. Overall, the hourly volatility of electricity returns in Ontario are much higher than in the other two markets. Hence, we conclude that Ontario electricity market has high volatility compared to PJM and Nord pool electricity markets.

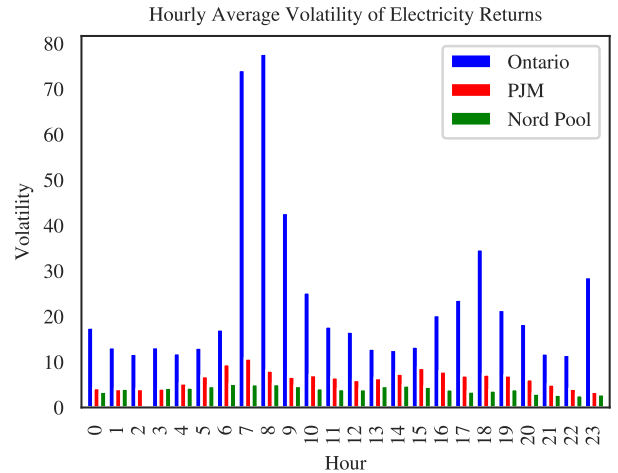


Fig. 3. Comparison of Ontario electricity market vs. PJM and Nord Pool markets in terms of volatility.

C. Day-Ahead Volatility Forecasts Used in TES Method

According to Fig. 3, electricity prices vary from hour to hour. Therefore, the DA volatility forecasts of electricity returns can be considered as a *feature for training the machine learning based EPF models*. We use the observed hourly volatility and TES method to compute one step-ahead forecasts of electricity returns volatility denoted by $\hat{\sigma}_{t+1,h}$ for all the subsets. The TES is a forecasting method that can capture trends and seasonality in a time series such as observed volatility. In section II-B, we calculated the observed hourly volatility of just one year. In order to obtain DA forecasts

of 2019 returns volatility we divide 2010-2019 HOEP data into 24 subsets each has N_o samples. The hourly observed volatility of electricity returns

$$s_{t,h} = \frac{|r_{t,h} - \bar{r}_h|}{\hat{\rho}_h}$$

is applied to triple exponential smoothing method to compute $\hat{\sigma}_{t+1,h}$. TES is a forecasting method that can capture trend and seasonality in the time series. This method considers a quadratic model for the data in each subset as

$$s_{t,h} = \beta_{0,h} + \beta_{1,h}t + \beta_{2,h}\frac{t^2}{2} + \epsilon_{t,h},$$

where $\epsilon_{t,h}$ refers to the white noise. The TES smoothed statistic can be obtained through the following steps:

Step I. Initialization:

$$\begin{aligned} S_{0,h}^1 &= \hat{\beta}_{0,h} - \frac{\omega_h}{\alpha_h} \hat{\beta}_{1,h} + \frac{\omega_h(2 - \alpha_h)}{2\alpha_h^2} \hat{\beta}_{2,h}, \\ S_{0,h}^2 &= \hat{\beta}_{0,h} - \frac{2\omega_h}{\alpha_h} \hat{\beta}_{1,h} + \frac{2\omega_h(3 - 2\alpha_h)}{2\alpha_h^2} \hat{\beta}_{2,h}, \\ S_{0,h}^3 &= \hat{\beta}_{0,h} - \frac{3\omega_h}{\alpha_h} \hat{\beta}_{1,h} + \frac{3\omega_h(4 - 3\alpha_h)}{2\alpha_h^2} \hat{\beta}_{2,h}, \end{aligned} \quad (2)$$

in which $\omega_h = 1 - \alpha_h$.

Step II. One-step-ahead forecast of volatility ($t \in [0, N_o]$)

$$\begin{aligned} S_{t,h} &= (3 + \frac{\alpha_h(6 - 5\alpha_h)}{2\omega_h^2} + \frac{\alpha_h^2}{\omega_h^2})S_{t,h}^1 \\ &\quad - (3 + \frac{\alpha_h(5 - 4\alpha_h)}{\omega_h^2} + \frac{\alpha_h^2}{\omega_h^2})S_{t,h}^2 \\ &\quad + (1 + \frac{\alpha_h(4 - 3\alpha_h)}{2\omega_h^2} - \frac{\alpha_h^2}{2\omega_h^2})S_{t,h}^3 \end{aligned} \quad (3)$$

Step III. Updating ($t \in [1, N_o]$)

$$\begin{aligned} S_{t+1,h}^1 &= (1 - \omega_h)s_{t+1,h} + \omega_h S_{t,h}^1 \\ S_{t+1,h}^2 &= (1 - \omega_h)S_{t+1,h}^1 + \omega_h S_{t,h}^2 \\ S_{t+1,h}^3 &= (1 - \omega_h)S_{t+1,h}^2 + \omega_h S_{t,h}^3 \end{aligned} \quad (4)$$

In these steps, $S_{t,h}$ is the smoothed value and also is the forecast of return volatility at time $t + 1$ for each subset. The tuning parameter is $\alpha_h \in [0, 1]$, and its optimal value can be obtained by minimizing the one-step-ahead forecast error sum of squares

$$\text{SSE}(\alpha_h) = \sum_{t=1}^{N_o} (s_{t,h} - S_{t-1,h})^2.$$

Finally, the optimum value of α_h is used in the all 3 steps to compute $\hat{\sigma}_{t,h}$.

III. DAY-AHEAD FORECASTS OF ELECTRICITY DEMAND

Electricity demand forecasting is a supervised learning process in which a regression method is applied to a set of features such as meteorological variables to predict the dependable variable. The common forms of analysis for this purpose are linear regression and decision trees. In this study, we employ gradient boosting regression methods.

A. Gradient Boosting Regression

Assume that we have a training data set $\{(x_i, y_i)\}_{i=1}^{N_s}$ and a differentiable loss function $\mathcal{L}(y_i, F(x_i))$ where x_i represents the i^{th} sample of the features in the training data set. And y_i is the variable to be predicted. The principle of gradient boosting regression for making trees is illustrated in algorithm (1) [15]. The algorithm (1) attempts to create a total of M trees. First, the GBR algorithm computes an initial predicted value such as γ for all the observations. Then the pseudo-residual r_{im} for all data in the training set are computed where m is the tree index. The algorithm uses feature data to build a regression tree with J_m leaves. Each tree categorizes r_{im} s into the leaves. As a result, each leaf has a subset of pseudo-residuals that is called terminal region and denoted by R_{jm} . The output value of each leaf, γ_{jm} , is computed for the leaves in the trees by minimizing a loss function. Eventually, the new prediction, $F_m(x_i)$, is derived from the previous prediction and output of the recent tree. ν is the learning rate. A small learning rate reduces the effect that each tree has on the final prediction.

Algorithm 1 Gradient Boost Regression

- 1: Initialize model with a constant value: $F_0(x) = \text{argmin}_{\gamma} \sum_{i=1}^{N_s} \mathcal{L}(y_i, \gamma)$
- 2: **for** $m = 1, 2, \dots, M$ and $i = 1, \dots, N_s$ **do**
- 3: Compute $r_{im} = -[\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)}]_{F(x)=F_{m-1}(x)}$
- 4: Fit a regression tree to the r_{im} values
- 5: **for** $j = 1, 2, \dots, J_m$ **do**
- 6: Create terminal regions R_{jm}
- 7: $\gamma_{jm} = \text{argmin}_{\gamma} \sum_{x_i \in R_{jm}} \mathcal{L}(y_i, F_{m-1}(x_i) + \gamma)$
- 8: **end for**
- 9: $F_m(x_i) = F_{m-1}(x_i) + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(x_i \in R_{jm})$
- 10: **end for**
- 11: Return $F_M(x_i), i = 1, \dots, N_s$.

A more sophisticated version of GBR is extreme gradient boosted (XGB) regression tree. The XGB regression uses the same loss function used in GBR to build trees by calculating the optimal output values of the j^{th} leaf in the m^{th} tree, γ_{jm} , so that

$$\gamma_{jm} = \text{argmin}_{\gamma} \sum_{x_i \in R_{jm}} \mathcal{L}(y_i, F_{m-1}(x_i) + \gamma) + \frac{1}{2} \lambda \gamma^2, \quad (5)$$

in which λ is the regularization term. XGB uses the second-order Taylor approximation to minimize the total loss function in eq. (5)

$$\mathcal{L}(y_i, F_{m-1}(x_i) + \gamma) \approx \mathcal{L}(y_i, F_{m-1}(x_i)) + G_i \gamma + \frac{1}{2} H_i \gamma^2,$$

where G_i (Gradient) and H_i (Hessian) are the first and second derivatives of the loss function. It can be proved that the optimal leaf output that minimizes the loss function is

$$\gamma_{jm} = -\frac{\sum_{x_i \in R_{jm}} G_i}{\sum_{x_i \in R_{jm}} H_i + \lambda}.$$

Other characteristics that make XGB a powerful tool for regression and classification are presented in [16].

B. Evaluation Metrics

We use mean absolute error (MAE), and root mean square error (RMSE) as evaluation criteria to compare the forecast errors of the models. Consider y_i to be the observations in the testing data set with N_t samples, and $\hat{y}_i$ to be the forecasts of the observations. Then

$$\text{MAE} = \frac{\sum_{i=1}^{N_t} |y_i - \hat{y}_i|}{N_t},$$

and

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{N_t} (y_i - \hat{y}_i)^2}{N_t}}. \quad (6)$$

Besides, we calculate R-squared (R^2) criteria to determine what portion of the variance in the data is explained by the forecasts data. The best possible score from these criteria is 1.0, and negative values are also possible, as

$$R^2 = 1 - \frac{\sum_{i=1}^{N_t} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N_t} (y_i - \bar{y})^2}, \quad (7)$$

where $\bar{y}$ indicates average of the observations in testing data.

C. Load Forecasting

The relationship between meteorological variables and electricity demand in Ontario has been previously studied in [17]. Hourly weather data such as weather temperature, precipitation, solar radiance, and wind speed can be downloaded from Canada's government website (climate.weather.gc.ca). We create a feature set including weather data as well as the hour of the day, day of the week, and day of the year. The weather and electricity demand data from 2015 until 2018 are considered as the training data, and the performance of the model is tested on 2019 data. We use the training data set to train GB and XGB regressors and finally predict DA electricity demand for 2019. Parameters of the two regressors are tuned in a grid search process. The MAE and RMSE of load predictions are provided in table II. An intuition about how high or low these metrics are, the mean and standard deviation of the electricity demand time series are 15485.36 and 2315.31, respectively. Our analysis results, particularly the high value of R^2 in table II, indicate that both regression methods perform well in predicting 2019 electricity demand in Ontario. However, the GB has slightly better predictions than XGB, and we use the GB's results for the next section.

TABLE II
EVALUATION OF XGB PREDICTION OF ELECTRICITY DEMAND IN ONTARIO.

Model	MAE	RMSE	R^2
GB Regression	367.41	480.74	0.953
XGB Regression	368.55	486.22	0.951

IV. ARTIFICIAL NEURAL NETWORKS FOR TIME SERIES FORECASTING

Artificial neural networks (ANN) are computing units created artificially to simulate biological neural networks. Input, hidden, and output layers are the building blocks of ANNs' structure. ANNs have applications in time series forecasting. Assume an ANN model that takes x_t as input vector to compute forecasts of y_t . The input layer passes x_t to a hidden layer with one neuron which applies its weights W_h and bias b_h , as well as an activation function $f_h(\cdot)$ (such as sigmoid, tanh or Relu) to generates h_t as the neuron's output

$$h_t = f_h(W_h^T x_t + b_h),$$

where T denotes transpose of the W_h . Assume that the output layer has just 1 neuron which receives h_t and computes $\hat{y}_t$ using its own weights W_o , bias b_o , and activation function $f_o(t)$ as

$$\hat{y}_t = f_o(W_o^T h_t + b_o).$$

The weights and biases of neurons are tuned as the learning process proceeds. The number of neurons and layers in the model is obtained using a trial and error method. Usually, the networks with more than two hidden layers are called deep networks and can capture important features in the downloaded HOEP data. Furthermore, neurons can make a dense layer when they are connected to all the neurons in the previous layer.

A. Recurrent Neural Networks (RNNs)

RNNs are another type of ANNs equipped with memory that enables them to apply the previous neuron's output to compute future outcomes. This particular feature makes them powerful for time series forecasting. The simple predictive model described in IV can be considered as a linear regression model. A non-linear forecasting model can apply by using a simple RNN model for HOEP series, which is almost the same as ANN, and the basic difference is the hidden state in RNN is a non-linear function of the input. That is

$$h_t = f_h(W_{xh}^T x_t + W_{hh}^T h_{t-1} + b_h), \quad (8)$$

where W_{xh}^T is a vector of weights connecting the input to the RNN neuron, and W_{hh}^T refers to the feedback link from the lagged value of RNN output to its input. At first glance, the simple RNN unit considers all the previous input data to calculate the model output. But, simple RNNs are very vulnerable to the vanishing gradient problem, and they have short memories. In other words, the vanishing gradient problem causes the RNN model to forget what it has learned from lagged values of the input. The most effective way to deal with this problem is to use a more sophisticated unit such as long short-term memory (LSTM) or GRU. GRU is like a simplified version of LSTM with fewer parameters and thus more efficient. Compare to the eq. (8), the hidden state for a GRU unit is more complicated and depends on update

gate vector z_t and reset gate vector r_t which can be written as

$$z_t = S(W_{xz}^T x_t + W_{hz}^T h_{t-1} + b_z),$$

and

$$r_t = S(W_{xr}^T x_t + W_{hr}^T h_{t-1} + b_r),$$

where S is a Sigmoid function and indicates that z_t and r_t vary between 0 and 1. The GRU hidden state h_t is expressed as

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}^T x_t + W_{hh}^T (r_t \odot h_{t-1} + b_h)) \quad (9)$$

where $\odot$ means element-wise multiplication as the two gates (r_t and z_t) and hidden vector have the same dimension. It follows from eq. (9) that if we neglect the reset gate, h_t turns out to be a weighted sum of the previous hidden state and output of a simple RNN. The role of the reset gate in eq. (9) is reinforcing the ability of the GRU unit to remember and forget different parts of h_{t-1} . LSTM units have almost the same architecture as GRUs, however they are different. In addition to the hidden state h_t , LSTMs have an extra state, c_t , which is called cell state. LSTM structure can be considered as the connection of some neurons to perform either as gates or generating cell states. The cell state

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_{xc}^T x_t + W_{hc}^T h_{t-1} + b_c), \quad (10)$$

where $f(t)$ is the forget gate given by

$$f_t = S(W_{xf}^T x_t + W_{hf}^T h_{t-1} + b_f),$$

and i_t is input/update gate given as

$$i_t = S(W_{xi}^T x_t + W_{hi}^T h_{t-1} + b_i).$$

Eq. (10) indicates that LSTM offers the cell state to have the opportunity to remember its old states so that the unit learns from long term dependencies. Eventually, the output gate o_t :

$$o(t) = S(W_{xo}^T x_t + W_{ho}^T h_{t-1} + b_o),$$

which is simple RNN unit (the same as the other two LSTM gates) is used to compute the hidden state through a simple transformation on the cell state

$$h_t = o_t \odot \tanh c_t. \quad (11)$$

B. Convolutional Neural Networks (CNNs)

Recurrent neural networks can use historical data to compute forecasts of future values. However, performance of the recurrent neurons declines as the dimension of the input data increases. Therefore, we need a pre-process step which extracts the important features from input data and map them into a low dimension space. CNNs can smooth the input data and extract features from them so that the next layers focus on more essential data with much less dimension. The input data which is a time series with length L , goes through some filters. The number of filters, N_F , is arbitrary. Each filter consists of l neurons with weights, biases and an activation function.

The input data is divided in some chunks named kernels each has length of l . In order to create a feature map, all of the kernels go through a filter. By changing weights and biases of the neurons we can make new filters and have a number of feature maps. Eventually, a long input vector with length L is changed to N_F smaller vector each has size l . The feature maps are proper input data for recurrent units because they are low dimensional vectors which carry important data.

V. POINT FORECASTS OF ELECTRICITY PRICES

Section II shows that electricity demand and renewable output are two explanatory variables for electricity prices. There is also a significant correlation between the HOEP time series and its lagged values [13]. This section uses the features to implement three forecasting models and eventually create a pool of point forecasts.

A. Convolutional Recurrent Neural Network

The first model used to compute day-ahead forecasts of HOEPs consists of Convolutional and Recurrent layers. Since recurrent neural networks use weights and biases between 0 and 1, the input data (HOEP data from 2010 until 2019) need to be scaled first. The 2019 HOEP data is used as the testing data. And, the rest of the data (2010-2018) is used for training and validation. Our analysis shows that almost 2.5 percent of the whole HOEP data are spike prices with the maximum and minimum of 1822.95 and -138.79 CAD/MWh (the negative price implies that electricity supply exceeds the demand and eventually suppliers have to pay their customers for selling the electricity). The majority of electricity prices vary between 10.00-33.00 CAD/MWh, and the outliers are unfavorable and almost cancel out the effect of scaling. In order to solve this problem, we set the negative and positive spikes to $Q_1 - 1.7 \times IQR$ and $Q_3 + 1.7 \times IQR$, respectively. The scaled HOEP time series, p_t^s , is obtained through a relatively simple transformation

$$p_t^s = \frac{p_t - p_{min}}{p_{max} - p_{min}},$$

in which p_t is the original HOEP data with minimum and maximum values p_{min} and p_{max} , respectively.

Another essential task in preparing data is to change the structure of the data set. The original training data is a 78888×1 dimensional column vector, but the Convolutional and recurrent layers expect 3-dimensional input data such as (Samples, Time Steps, Features). Here, each sample refers to electricity prices data for 168 hours. Time steps refer to the number of historical electricity prices that our model uses to forecast the next 24 hourly prices. For instance, using the last seven days (168 hours) prices to forecast future 24 hours would require the training input and output data to have (3280, 168, 1) and (3280, 24) dimensions, respectively.

The first forecasting model consists of different layers. First, a convolution layer receives sample data from the input layer and extracts the important features from the data. The next layer uses one of the recurrent units such as simple RNN,

GRU, or LSTM to see which one has better performance. As a matter of fact, the performance of recurrent neurons get worse as the dimension of their input data increases. The CNNs have the ability to extract the main features of the data and do the dimension reduction. As a result, the neurons in recurrent layer (come after convolution layer) receive low dimensional data. The output of recurrent layer goes to a fully connected units of a dense layer to generate the DA forecasts of the electricity prices. The training data set is HOEPs from 2010 to 2018, 20 percent of which is taken as validation data. In order to avoid overfitting, we have added a dropout layer after the recurrent layer to mask 10 percent of neurons in each training. The performance of the model is tested on an out of sample 2019 HOEP data. The model is implemented using Keras library and Tensorflow backend in Python. The parameters of the model are presented in table III. A brief description on Adam algorithm and corresponding parameters is provided in Appendix section.

TABLE III
INITIAL VALUES FOR THE ANN FORECASTING MODEL

Parameters	Value
Convolution Layer Filters	256
Convolution Layer Kernel Size	24
Numbers of Recurrent Units	256
Dropout Rate	0.10
Number of Dense Layer Units	24
Number of Lagged Observation	168
Optimizer	Adam
Batch Size	80
Epoch	200
Learning Rate	0.001
Loss Function	Mean Squared Error

B. XGB Forecasts

The second model employs decision tree regression methods. So far, we have computed the DA forecasts of electricity load and electricity returns volatility. Hence we can have more accurate results for point forecasts of HOEP by using the two features. GB and XGB regressors are assigned to predict HOEPs, using a set of explanatory variables from 2015 until 2018 that includes

- Historical HOEP for the last 168 hours,
- Day-ahead forecasts of electricity return volatility, and load,
- Time/date related variables such as time of the day, day of the week, and day of the year.

The actual 2015-2018 HOEPs are considered as the dependant variable. The regressors are trained using the 2015-2018 features data set and tuned in a grid search process. Our analysis shows that the XGB model outperforms GB by decreasing the ConvGRU model's MAE from 8.20 to 7.06. It should be mentioned that using the DA volatility forecasts as a feature improves the XGB performance by 15.79 percent. In other words, without DA volatility forecasts, the XGB regressor results with a MAE of 7.24.

C. Triple Exponential Smoothing

The third method considers the same idea that is used for volatility forecasting. In more details, the 2010-2019 HOEP data break to 24 subsets in a way that each subset contains electricity prices for a particular hour. Then, triple exponential smoothing method computes DA forecasts of 2019 HOEP data.

D. Benchmark Forecasting Model

In order to determine how well the ANN models perform in computing DA forecasts of electricity prices, we need to set up benchmark models to compare the results. Benchmarks are simple models that perform forecasting based on the best guess. We define four benchmark models which consider the forecast of electricity prices for each hour of the next day as:

- **B1:** price at the same hour of the previous day,
- **B2:** price at the same hour and day of previous week,
- **B3:** average prices at the same hour and day during last 2 weeks,
- **B4:** average price at the same hour and day during last 4 weeks.

Table IV presents a comparison of the performance of forecasting models against benchmarks B1-B4. It reveals that among all of the forecasting models the ConvGRU results in the most accurate predictions. Performances of the three

TABLE IV
COMPARISON OF FORECASTING MODELS AGAINST BENCHMARKS

Model	MAE	RMSE
Benchmark1	11.26	29.31
Benchmark2	12.88	30.12
Benchmark3	10.82	25.65
Benchmark4	10.30	23.68
Triple Exponential Smoothing	9.35	21.62
ConvRNN Model	8.64	21.34
ConvLSTM Model	8.38	21.06
ConvGRU Model	8.22	21.03
GB Regression	7.15	20.02
XGB Regression	7.06	20.01

forecasts are compared in Fig. 4 in terms of hourly MAE.

VI. PROBABILISTIC FORECASTS OF ELECTRICITY PRICES

The most common way to generate probabilistic forecasts of electricity prices is to compute the PIs of point forecasts of electricity prices. The point forecast for electricity price at time t , i.e., $\hat{p}_t$, and the forecasting error

$$\epsilon_t = p_t - \hat{p}_t$$

are the two main elements for computing PIs. Basically, the center of PIs is set to $\hat{p}_t$ and cumulative distribution function (CDF) of ϵ_t , F_{ϵ_t} , is used to derive lower and upper bounds of PIs. For instance, 5% and 95% quantiles of ϵ_t are required to compute 90% PI of electricity price. The idea of computing PIs can be extended further to calculate 99 quantiles ($q = 1\%, \dots, q = 99\%$) as

$$F_{p_t}^{-1}(q) = \hat{p}_t + F_{\epsilon_t}^{-1}(q)$$

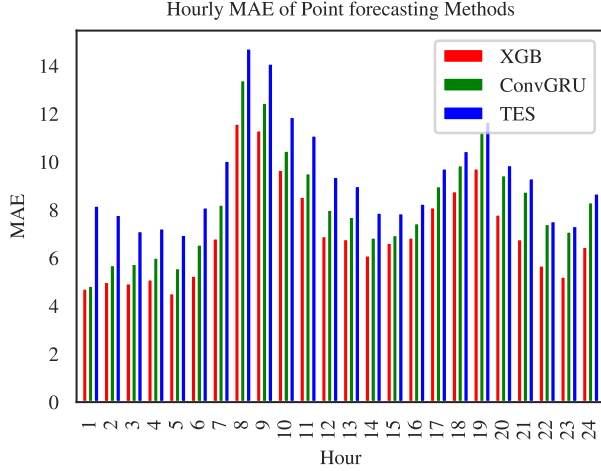


Fig. 4. MAE of 3 Different Day-ahead Point Forecasting Models.

and obtain a discretization of the electricity price distribution. Different methods have been proposed to calculate the distribution of the error term. QRA is a method to derive quantiles of the electricity price distribution. Initially, the QRA method considers point forecasts of electricity prices as explanatory variables for the quantile of the electricity price distribution. The number of point forecasts can be chosen arbitrarily. During the GEFCom2014, some of the exogenous variables such as hourly electricity load/price forecasts were added to the explanatory variables, which eventually improved the QRA method's results.

The QRA method computes conditional q^{th} quantiles of electricity price distribution $Q_{p_t}(q | \cdot)$ as

$$Q_{p_t}(q | X_t) = f(X_t),$$

where X_t is a vector of explanatory variables including point forecasts and $f(\cdot)$ is a transformation function [8]. The optimum $f(\cdot)$ minimizes the loss function

$$\mathcal{L}(p_t, q) = \sum_{t=1}^{N_t} (q - \mathbf{1}_{p_t < f(X_t)})(p_t - f(X_t)).$$

The quantile regression method models $f(\cdot)$ as a linear function. However, $f(\cdot)$ can be a non-linear function such as decision trees or neural networks. We use QRA, GBR and NN methods to minimize the loss function and eventually derive the prediction intervals of day-ahead electricity prices. Fig. 5 shows the total quantile loss for each of the three methods. The figure reveals that when $q \leq 0.5$, QRA minimizes the loss function better than the other two methods. However, as q increases, GB shows better performance than the other methods.

One of the criteria that we use to compare the results of 3 models in quantile forecasts is reliability. Reliability determines how the quantile forecasts $\hat{q}_t^{(\alpha)}$ at the α nominal level match with empirical findings. For instance, when $\alpha = 0.9$

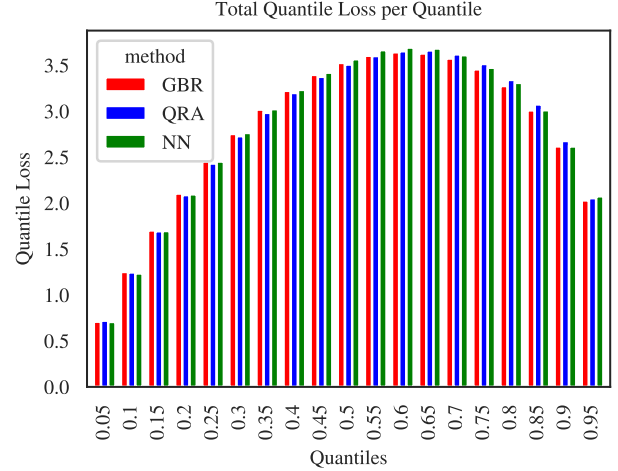


Fig. 5. Total loss per quantile of predictive models.

one may expect that 90 percent of the electricity prices in our data set to be less than $\hat{q}_t^{(0.9)}$. The empirical level of the quantile forecasts is given by

$$a_\alpha = \frac{\sum_{t=1}^{N_t} \mathbf{1}(p_t < \hat{q}_t^{(\alpha)})}{N_t}.$$

A set of empirical levels of quantile forecasts can be obtained for different values of α in $[0,1]$. We call the mean squared deviation of empirical results and nominal levels as reliability deviation. Among various quantile forecasters, the most accurate one has the least reliability deviation. To determine how tight are the prediction intervals, we employ the sharpness metric. The sharpness of interval forecasts is obtained by calculating their average of the differences of the predicted quantiles

$$s_\delta = \frac{\sum_{t=1}^{N_t} \hat{q}_t^{(\alpha_u)} - \hat{q}_t^{(\alpha_l)}}{N_t}$$

where α_u and α_l are the upper and lower bounds, and δ is the difference between them. The smallest sharpness value indicates that the corresponding quantile forecasts method has the narrowest PIs. In order to evaluate the overall performance for a probabilistic forecast model, we compute the most common skill score for predictive densities known as continuous ranked probability score (CRPS). In fact, the CRPS generalizes the MAE to the case of probabilistic forecasts. For a given predictive density of electricity prices, $\hat{F}_{p_t}(x)$ and corresponding observation p_t

$$\text{CRPS}(\hat{F}_{p_t}(x), p_t) = \int_{-\infty}^{\infty} (\hat{F}_{p_t}(x) - \mathbf{1}_{\{p_t \leq x\}})^2 dx. \quad (12)$$

However, the integral in eq. (12) encompasses numerical difficulties. A more convenient way to compute CRPS is to use the loss function, so that

$$\text{CRPS}(\hat{F}_{p_t}(x), p_t) = \int_0^1 \mathcal{L}(p_t, q) dq.$$

Finally, the CRPS score value is given by taking average for each of the predictive densities in the testing data set

$$CRPS = \frac{1}{N_t} \sum_{t=1}^{N_t} CRPS(\hat{F}_{p_t}(x), p_t).$$

Table V compares the three methods and indicates that the GBR method has the least score in all metrics. It means that GBR quantile forecasts cover more observation than other methods while it has the tightest PIs. In addition, the CRPS for GBR indicates that GBR quantile forecasts are more accurate than the other two methods.

TABLE V
EVALUATION OF PREDICTIVE DENSITY MODELS.

Model	Reliability	Deviation	Sharpness	CRPS
Quantile Regression	0.008		11.30	1226.4
Neural Network	0.058		11.17	1270.2
Gradient Boosted	0.002		11.12	1182.6

The results of point and probabilistic forecasts of hourly electricity prices for a random period in 2019 in Fig. 6. The red squares are the actual electricity prices, and the solid blue line shows the point forecasts obtained from the XGB regressor. We can see that our point forecasts follow the trends in actual prices, although there are uncertainties in the forecasting results. As illustrated in the figure, the PIs give more information about future electricity prices and compensate for point forecasts' uncertainty to some extent.

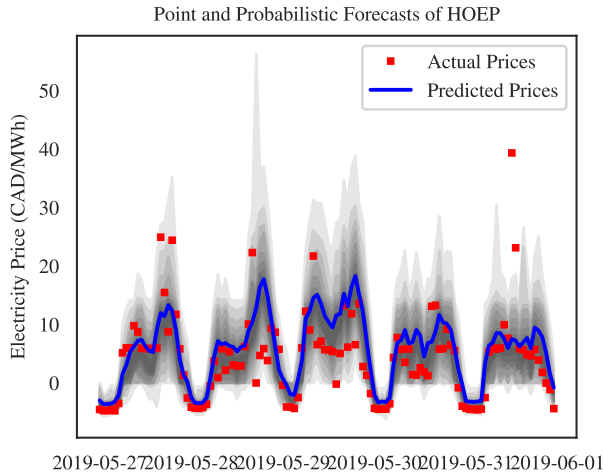


Fig. 6. Probabilistic, Point, and Quantile Forecasts of Ontario Electricity Prices.

VII. CONCLUSION

This paper computes quantile forecasts of day-ahead electricity prices in the Ontario electricity market. A pool of point forecasts is made by applying three different methods. The TES method uses a relatively simple idea to break the HOEP

data set into 24 subsets. Each subset has electricity prices of a particular hour. The TES method applies to each subset and generates point forecasts of the corresponding hour. The TES method decreases MAE of a benchmark model based on the 2019 HOEP data from 10.29 to 9.42. The ConvGRU model uses HOEP data in order to compute the DA point forecasts of electricity prices and results an MAE of 8.20. The last method uses the extreme gradient boosted regression approach and obtains another set of DA point forecasts. The XGB method uses explanatory variables such as historical HOEP, and predictions of electricity demand and return volatility. The XGB regression approach further decreases the MAE to 7.06. The predictions of electricity demand are obtained by GBR which uses historical meteorological variables of Ontario as input. Moreover, forecasts of electricity return volatility are computed by the TES method. Finally, QRA, GBR and NN methods are applied to the pool of point forecasts to compute quantile forecasts of DA electricity prices. Our analysis shows that the GBR method results in more reliable quantiles and tighter PIs with smaller CRPS than QRA.

REFERENCES

- [1] Zhang, Li et al. "Energy clearing price prediction and confidence interval estimation with cascaded neural networks". *IEEE Transactions on Power Systems* 18. 1(2003): 99–105.
- [2] Zhang, Li et al. "Neural network-based market clearing price prediction and confidence interval estimation with an improved extended Kalman filter method". *IEEE Transactions on Power Systems* 20. 1(2005): 59–66.
- [3] Khosravi, Abbas et al. "A neural network-GARCH-based method for construction of Prediction Intervals". *Electric Power Systems Research* 96. (2013): 185–193.
- [4] Khosravi, Abbas et al. "Quantifying uncertainties of neural network-based electricity price forecasts". *Applied energy* 112. (2013): 120–129.
- [5] Chen, Xia et al. "Electricity price forecasting with extreme learning machine and bootstrapping". *IEEE Transactions on Power Systems* 27. 4(2012): 2055–2062.
- [6] Wan, Can et al. "A hybrid approach for probabilistic forecasting of electricity price". *IEEE Transactions on Smart Grid* 5. 1(2013): 463–470.
- [7] Gaillard, Pierre et al. "Additive models and robust aggregation for GEFCom2014 probabilistic electric load and electricity price forecasting". *International Journal of forecasting* 32. 3(2016): 1038–1050.
- [8] Nowotarski, Jakub et al. "Computing electricity spot price prediction intervals using quantile regression and forecast averaging". *Computational Statistics* 30. 3(2015): 791–803.
- [9] Maciejowska, Katarzyna et al. "A hybrid model for GEFCom2014 probabilistic electricity price forecasting". *International Journal of Forecasting* 32. 3(2016): 1051–1056.
- [10] Dudek, G. (2016). Dudek, Grzegorz. "Multilayer perceptron for GEFCom2014 probabilistic electricity price forecasting". *International Journal of Forecasting* 32. 3(2016): 1057–1060.
- [11] Hubicka, Katarzyna et al. "A note on averaging day-ahead electricity price forecasts across calibration windows". *IEEE Transactions on Sustainable Energy* 10. 1(2018): 321–323.
- [12] Serafin, Tomasz et al. "Averaging predictive distributions across calibration windows for day-ahead electricity price forecasting". *Energies* 12. 13(2019): 2561.
- [13] Zareipour, Hamidreza et al. "Electricity market price volatility: The case of Ontario". *Energy policy* 35. 9(2007): 4739–4748.
- [14] Thavaneswaran, Aerambamoorthy et al. "Generalized value at risk forecasting". *Communications in Statistics-Theory and Methods* 49. 20(2020): 4988–4995.
- [15] Friedman, Jerome H. "Stochastic gradient boosting". *Computational statistics and data analysis* 38. 4(2002): 367–378.
- [16] Chen, Tianqi et al. "Xgboost: A scalable tree boosting system." *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*.

- [17] Banitalebi, Behrouz et al. "Modeling of Short-Term Electricity Demand and Comparison of Machine Learning Approaches for Load Forecasting." 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC).

APPENDIX

A. Gradient Descent

Training algorithms for ANNs seek to find the optimal weights and biases for the neurons that minimize a loss function which represents the deviation of predicted values from the actual ones. For example, mean squared error (MSE) can be considered as the loss function and is defined for neurons' weights as

$$\mathcal{L}(W) = \frac{1}{N_s} \sum_{t=1}^{N_s} (y_t - \hat{y}_t)^2.$$

Gradient descent is an optimization algorithm that computes the derivative of the loss function and then repeatedly take small steps in the direction of the gradient. Eventually, gradient descent algorithm converge to the minimum. Epoch is the number of iterations that algorithm should run. Moreover, the learning rate η , refers to size of steps that the algorithm takes toward the minimum of the loss function. When the size of training data is large, computing the loss function for all the data takes a lot of time. The solution for the problem is stochastic gradient descent algorithm which uses a randomly selected batch of the data at every step rather than the full data.

Momentum is a significant improvement over the plain stochastic gradient descent and can improve the learning procedure. The regular gradient descent without momentum is expressed as

$$W_t = W_{t-1} - \eta G_t, \quad (13)$$

where G_t refers to gradient of the loss function. When $G_t = 0$, there is no update for neurons' weights. In order to model gradient descent with momentum, we add a velocity term V_t in equation (13) so that

$$V(t) = \mu V_{t-1} - \eta G_t,$$

and neurons weights are updated as

$$W_t = W_{t-1} + V_t,$$

where μ is the momentum. Typical values for μ are between 0.90 and 0.99 which show the fact that we want the new velocity has a fraction of old velocity. Momentum helps the training procedure to converge to optimal neurons' weights much faster than regular gradient descent.

B. Variable and Adaptive Learning Rates

Variable learning rate can speed up the convergence of gradient descent algorithm. Adaptive Gradient Algorithm (AdaGrad) is based on the fact that the loss function on each neuron weight is not the same. AdaGrad introduces a new variable called cache and is shown by C_t . Each parameter of the neural network has its own cache, hence C_t has the same dimension

as W_t . The idea behind the cache is that it accumulates the squared gradients as

$$C_t = C_{t-1} + G_t \odot G_t,$$

in which $\odot$ represents element-wise multiplication. If a parameter has large gradient in the past then its cache becomes very large which leads to a very small learning rate. AdaGrad updates weights of neural networks as

$$W_t = W_{t-1} - \frac{\eta}{\sqrt{C_t} + \epsilon} \odot G_t, \quad (14)$$

where ϵ is a small value around 10^{-8} or 10^{-10} and is used to avoid division by zero.

The main drawback of AdaGrad is the fact that the cache is raising too fast and consequently the learning rate decreases aggressively. Root Mean Square Propagation (RMSProp) method which is an extension of AdaGrad restricts growth of the cache by decreasing it at each update as

$$C_t = \lambda \times C_{t-1} + (1 - \lambda) \times G_t \odot G_t, \quad (15)$$

where λ is the decay rate and is a number close to 1 such as 0.99, 0.999, etc. Eventually, RMSProp uses eq. (14) to update neuron weights. According to eq. (15), cache can be interpreted as the estimation of the expected value of the squared gradient,

$$C_t \approx E(G_t \odot G_t).$$

Adaptive moment estimation method (also called Adam) uses m_t and s_t as the 1st and 2nd moments of G_t in order to find optimal values for learning rates. The exponentially smoothed averages of m_t and s_t can be written as

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) G_t,$$

and

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) G_t \odot G_t,$$

in which β_1 and β_2 are coefficients. Adam employs the same cache mechanism that is used in RMSProp. The exponentially-smoothed average method needs an initial value for C_0 to calculate updates for C_t . Since there is no C_t at $t = 0$ we typically set the initial value to zero. By doing so, C_t is biased toward zero. Adam uses bias correction for exponentially smoothed averages of m_t and s_t as

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$

and

$$\hat{s}_t = \frac{s_t}{1 - \beta_2^t}.$$

Again, ϵ is a small number such as 10^{-8} and is used to avoid division by zero. Eventually, the Adam update of neurons' weights can be formulated as

$$W_t = W_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}.$$